02 - Control de flujo

Pregunta número 1

¿Cuál de las siguientes afirmaciones sobre el ciclo for mejorado introducido en Java 5 es verdadera?

Seleccione una:

- a. El ciclo for mejorado solo se puede utilizar con matrices.
- b. El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz.
- c. El ciclo for mejorado permite la modificación de los elementos de la colección durante la iteración.
- d. El ciclo for mejorado no requiere que la colección implemente la interfaz Iterable.

Respuesta: El ciclo for mejorado (también conocido como for-each) se puede utilizar con cualquier tipo de colección y matriz. Este ciclo for mejorado fue introducido para simplificar la iteración sobre arreglos y colecciones. El ciclo for mejorado funciona con dos tipos de estructuras: En arreglos (matrices) se puede recorrer cualquier arreglo (de tipos primitivos o de objetos) y en colecciones se puede recorrer cualquier objeto que implemente la interfaz java.lang.Iterable. Esto incluye prácticamente todas las colecciones estándar de Java, como List, Set, Queue, etc.

Pregunta número 2

What is the result value of w?

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 37;
        int z = 0;
        int w = 0;

        if (a == b) {
            z = 3;
        } else if (a > b) {
            z = 6;
        }

        w = 10 * z;

        System.out.println("El valor de w es: " + w);
    }
}
```

Seleccione una:

- a. 30
- b. 60
- c. 0

Respuesta: En este ejercicio hay que ir paso a paso por el código para determinar el valor final de w. Comencemos con la inicialización de variables: int a = 10; int b = 37; int z = 0; int w = 0. De ahí nos vamos a la evaluación de las condiciones if y else if: El if (a == b): ¿Es 10 igual a 37? No, es falso. Seguimos con el else if (a > b): ¿Es 10 mayor que 37? No, también es falso. Dado que ninguna de las condiciones se cumple, el bloque de código dentro del else if (z = 6;) no se ejecuta. Por lo tanto, la variable z conserva su valor inicial, que es 0. Ahora sí, vamos con el cálculo de w: La línea w = 10 * z; se ejecuta con el valor actual de z (que es 0), entonces w = 10 * 0; por lo tanto, w es 0. Finalmente, el programa imprime el valor de w, que es 0.

Pregunta número 3

Que muestra en pantalla el siguiente código?

```
public class LoopQuestion {
    public static void main(String[] args) {
        int count = 0;
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

Seleccione una:

- a. Compilation fails.
- b. Count: 5
- c. Count: 4
- d. Count: 10
- e. Count: 6

Respuesta: En este fragmento de código se evalúa el funcionamiento de un ciclo for en combinación con el operador módulo (%), el cual calcula el residuo de una división matemática. Primero, se inicializa la variable count en 0. El ciclo for declara una variable i que comienza en 0 y se incrementa de uno en uno mientras sea menor estrictamente que 10 (i < 10). Esto significa que i tomará exactamente los valores del 0 al 9. Dentro del ciclo, la instrucción if (i % 2 == 0) evalúa si el valor actual de i es par, verificando si el residuo de dividir i entre 2 es cero. La variable count se incrementa (count++) únicamente cuando la condición anterior es verdadera. En el rango del 0 al 9, los valores que cumplen esta condición de ser pares son: 0, 2, 4, 6 y 8. Dado que la condición se cumple exactamente en 5 iteraciones distintas, count acumula un valor final de 5. Al terminar el bucle, se imprime Count: 5.

Pregunta número 4

¿Cuál es la salida del programa?

```
public class NestedLoopTest {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("i."+i + "," + "j."+j);
            }
        }
    }
}
```

Seleccione una:

a.
```
i.1,j.1
i.1,j.2
i.2,j.1
i.2,j.2
i.3,j.1
i.3,j.2
```

b.
```
i.1,j.1
i.2,j.2
i.3,j.3
```

c.
```
i.1,j.1
i.1,j.2
i.1,j.3
i.2,j.1
i.2,j.2
i.2,j.3
```

d.
```
i.1,j.1
i.2,j.1
i.3,j.1
```

Respuesta: En este código estamos evaluando el comportamiento secuencial de los bucles anidados (nested loops) en Java. El bucle exterior controlado por la variable i se ejecuta tres veces, tomando los valores 1, 2 y 3. Por cada iteración individual del bucle exterior, el bucle interior controlado por j se inicializa desde cero y se ejecuta por completo, tomando los valores 1 y 2. Como resultado, la instrucción de impresión interna se evalúa un total de $3 \times 2 = 6$ veces. Siguiendo el flujo de ejecución paso a paso, se genera la salida exacta de la opción a: Cuando i=1: el bucle interno corre para j=1 imprimiendo i.1,j.1, y luego para j=2 imprimiendo i.1,j.2. Cuando i=2: el bucle interno corre para j=1 imprimiendo i.2,j.1, y luego para j=2

imprimiendo i.2,j.2. Cuando i=3: el bucle interno corre para j=1 imprimiendo i.3,j.1, y luego para j=2 imprimiendo i.3,j.2.

Pregunta número 5

¿Cuántas veces se ejecutará la instrucción println más interna en el siguiente código?

```
public class NestedForLoopTest {
    public static void main(String[] args) {
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 3; j++) {
                for (int k = 0; k < 2; k++) {
                    System.out.println("Hello");
                }
            }
        }
    }
}
```

Seleccione una:

- a. 6
- b. 12
- c. 24
- d. 36

Respuesta: En este ejercicio se está evaluando el comportamiento multiplicativo de los bucles anidados en Java. Para calcular el número total de veces que se ejecuta la instrucción más interna, basta con multiplicar la cantidad de iteraciones de cada bucle individualmente: El bucle exterior (i) inicia en 0 y se ejecuta mientras sea menor a 4, resultando en 4 iteraciones (0, 1, 2, 3). El bucle intermedio (j) inicia en 0 y se ejecuta mientras sea menor a 3, resultando en 3 iteraciones (0, 1, 2) por cada ciclo del bucle exterior. El bucle interior (k) inicia en 0 y se ejecuta mientras sea menor a 2, resultando en 2 iteraciones (0, 1) por cada ciclo del bucle intermedio. Al anidar estos bucles, las iteraciones se multiplican: 4 * 3 * 2 = 24 ejecuciones totales para la instrucción System.out.println("Hello").

Pregunta número 6

What is the result?

```
public class DoWhile {
    public static void main(String[] args) {
        int ii = 2;
        do {
            System.out.println(ii);
        } while (ii > 0);
    }
}
```

Seleccione una:

- a. 1 0
- b. 2 1 2
- c. Compilation fails
- d. An infinite loop

Respuesta: En este código sucede algo muy curioso: por un lado, va a compilar correctamente, pero también va a entrar en un bucle infinito en tiempo de ejecución. Para entender por qué está sucediendo esto, vamos a ir paso a paso: Se declara la variable int ii = 2. El bloque do se ejecuta al menos una vez. Se imprime el valor actual de ii, que es 2. Luego, se evalúa la condición del while: ii > 0. Como ii vale 2, la condición es verdadera. Al ser verdadera, el bucle vuelve a ejecutarse. El problema viene dentro del bloque do, ya que no hay ninguna instrucción que modifique el valor de ii (como ii-- o ii = ii - 1). Por lo tanto, cada vez que se evalúa la condición ii > 0, el valor de ii sigue siendo 2, lo que hace que la condición siempre sea verdadera. El programa se quedará atrapado imprimiendo 2 en la pantalla indefinidamente.

Pregunta número 7

What is the result?

```
public class DoWhile {
    public static void main(String[] args){
        int ii = 2;
        do{
            System.out.println(ii);
        }while(--ii);
    }
}
```

Seleccione una:

- a. 1 0
- b. Compilation fails.
- c. 2 1 2
- d. null
- e. An infinite loop

Respuesta: En este ejercicio se evalúa una regla fundamental de tipos en Java. A diferencia de lenguajes como C o C++, en Java las condiciones de las estructuras de control (if, while, for) deben ser estrictamente de tipo booleano. No se permite usar un número entero (int) como si fuera un booleano, donde el 0 sería false y cualquier otro número sería true. Veamos paso a paso qué está pasando en el código: Tenemos la declaración int ii = 2. El bloque do se ejecutaría normalmente (imprimiendo 2). Sin embargo, la condición del while es --ii. La expresión --ii es una operación aritmética que devuelve un valor de tipo int (en este caso, primero 1 y luego 0). Entonces llega el error de compilación: El compilador de Java espera encontrar una expresión de tipo boolean dentro de los paréntesis del while. Al encontrar un int, el compilador lanza un error de compilación del tipo: "incompatible types: int cannot be converted to boolean", que básicamente nos indica que los tipos son incompatibles, ya que int no se puede convertir a boolean. Por lo tanto, el programa ni siquiera llega a ejecutarse.

Pregunta número 8

What value should replace kk in line x to cause jj=5 to be output?

```
public class MyFive {
    public static void main(String[] args) {
        short ii;
        short jj = 0;
        for (ii = kk; ii > 6; ii -= 1) { // line x
            jj++;
        }
        System.out.println("jj=" + jj);
    }
}
```

Seleccione una:

- a. -1
- b. 1
- c. 5
- d. 8
- e. 11

Respuesta: En este ejercicio, la importancia recae en la lógica de control de los bucles for y la iteración mediante decrementos. El código requiere que la variable jj termine imprimiendo un valor de 5. Como jj se inicializa en 0 y se incrementa de uno en uno (jj++), el interior del bucle debe ejecutarse exactamente 5 veces. La condición del bucle es que el bucle continuará iterando mientras la variable ii sea estrictamente mayor que 6 (ii > 6), y al final de cada ciclo su valor se reduce en 1 (ii -= 1). Para que el bucle dé 5 vueltas exactas y se detenga al llegar a 6, la variable ii debe tomar secuencialmente los siguientes valores válidos para la condición: Vuelta 1: ii = 11 (jj pasa a 1). Vuelta 2: ii = 10 (jj pasa a 2). Vuelta 3: ii = 9 (jj pasa a 3). Vuelta 4: ii = 8 (jj pasa a 4). Vuelta 5: ii = 7 (jj pasa a 5). Cuando ii se reduce a 6, la condición 6 > 6 se evalúa como falsa, el bucle termina y el programa imprime el valor acumulado. Por lo tanto, para lograr este comportamiento, el valor de inicio kk debe ser 11.

Pregunta número 9

¿Qué imprime este bucle?

```
public static void main(String[] args) {
    int num = 0;
    int arr[] = {1,2,3,4,5};
    for(int i = 0; i < arr.length; i++) {
        if(i % 2 == 0) {
            num += arr[i];
        } else {
            num -= arr[i];
        }
    }
    System.out.println(num);
}
```

Seleccione una:

- a. 0
- b. 3
- c. 5
- d. -3

Respuesta: En este ejercicio se está evaluando la iteración sobre arreglos y el uso del operador módulo (%) para alternar el flujo de control entre sumas y restas dependiendo de si el índice es par o impar. Realizando el seguimiento de la ejecución paso a paso, tenemos primero que el estado inicial de la variable num comienza en 0 y el arreglo arr contiene los valores {1, 2, 3, 4, 5} alojados en los índices del 0 al 4. En la iteración i = 0, la condición 0 % 2 == 0 es verdadera (0 es par). Se ejecuta num += arr[0]. num pasa de 0 a 1. En la iteración i = 1, la condición 1 % 2 == 0 es falsa. Se ejecuta num -= arr[1]. num pasa de 1 a -1 (1 - 2). En la iteración i = 2, la condición 2 % 2 == 0 es verdadera. Se ejecuta num += arr[2]. num pasa de -1 a 2 (-1 + 3). En la iteración i = 3, la condición 3 % 2 == 0 es falsa. Se ejecuta num -= arr[3]. num pasa de 2 a -2 (2 - 4). En la iteración i = 4, la condición 4 % 2 == 0 es verdadera. Se ejecuta num += arr[4]. num pasa de -2 a 3 (-2 + 5). Al terminar esta iteración, el índice i alcanza el valor de 5, lo cual rompe la condición i <

arr.length (5 no es menor que 5) y el ciclo finaliza. La instrucción System.out.println(num); imprime el valor acumulado final, que es 3.

Pregunta número 10

What is the result if the integer value is 33?

```
public static void main(String[] args) {
    int value = 33;

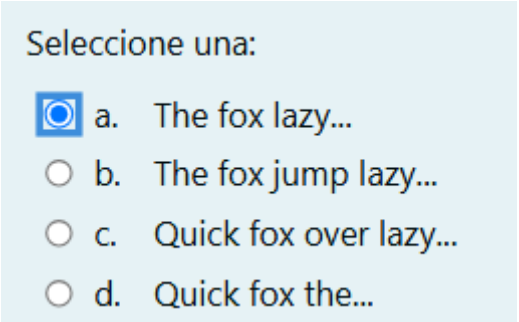
    if (value >= 0) {
        if (value != 0) {
            System.out.println("the ");
        } else {
            System.out.println("quick ");
        }

        if (value < 10) {
            System.out.println("brown ");
        }

        if (value > 30) {
            System.out.println("fox");
        } else if (value < 50) {
            System.out.println("jumps ");
        } else if (value < 10) {
            System.out.println("over ");
        } else {
            System.out.println("the ");
        }

        if (value > 10) {
            System.out.println("lazy ");
        } else {
            System.out.println("dog ");
        }

        System.out.println("...");
    }
}
```



Respuesta: En este tipo de ejercicio, la clave es notar que la mayoría de los bloques if son estructuralmente independientes, por lo que el programa evalúa la misma variable repetidas veces. Es importante resaltar que todas las instrucciones utilizan System.out.println(), lo que inserta un salto de línea después de cada evaluación exitosa. Analizando el flujo de ejecución con value = 33, tenemos que en el primer bloque, la condición externa (value >= 0) es verdadera. Dentro, la condición anidada (value != 0) también es verdadera, por lo que se ejecuta la impresión de "the ". En el segundo bloque, la condición (value < 10) evalúa a falso, por lo que este bloque se ignora por completo. En el tercer bloque, la condición (value > 30) es verdadera, imprimiendo "fox". Dado que las siguientes condiciones están encadenadas mediante else if, el programa las omite sin evaluarlas tras encontrar la primera coincidencia. En el cuarto bloque se inicia una nueva evaluación independiente. La condición (value > 10) es verdadera, lo que resulta en la impresión de "lazy". En la impresión final tenemos que la línea System.out.println("..."); se encuentra fuera de todas las estructuras condicionales, por lo que se ejecuta de forma incondicional al final del programa.

Pregunta número 11

What is the result?

```
public static void main(String[] args) {
    String color = "Red";
    switch (color) {
    case "Red":
        System.out.println("Found Red");
    case "Blue":
        System.out.println("Found Blue");
    case "White":
        System.out.println("Found White");
        break;
    default:
        System.out.println("Found Default");
    }
}
```

Seleccione una:

- a. Found Red.
- b. Found Red Found Blue.
- c. Found Red Found Blue Found White.
- d. Found Red Found Blue Found White Found Default.

Respuesta: En este ejercicio se evalúa el comportamiento de "fall-through" (caída en cascada) característico de las estructuras switch tradicionales en Java. La variable color se inicializa con el valor "Red". El bloque switch evalúa esta variable y salta directamente al punto de entrada coincidente: En el case "Red" se ejecuta la primera instrucción y se imprime "Found Red". Dado que no existe una instrucción break; al final de este caso, el flujo de ejecución no sale del bloque switch, sino que "cae" secuencialmente al siguiente bloque (case "Blue":), ignorando su condición, e imprime "Found Blue". Nuevamente, al faltar un break, la ejecución continúa cayendo hacia el case "White", imprimiendo "Found White". Es en este punto donde el flujo encuentra finalmente una instrucción break; esto interrumpe la cascada, forzando la salida del switch e impidiendo que se llegue a ejecutar el bloque default.

Pregunta número 12

What is the result?

```
enum Days {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY
}
public class Test {
    public static void main(String[] args) {
        int y = 1;
        Days d = Days.TUESDAY;
        switch (d) {
            case MONDAY: y++;
            case TUESDAY: y += 10;
            case WEDNESDAY: y += 100;
            case THURSDAY:  y += 1000;
        }
        System.out.println(y);
    }
}
```

Seleccione una:

- a. 11
- b. 111
- c. Compilation fails.
- d. An exception is thrown at runtime.
- e. 1111 (seleccionada)

Respuesta: En este ejercicio se nos vuelve a presentar el comportamiento de "fall-through" (caída en cascada) en las estructuras switch, esta vez aplicado sobre un tipo enumerado (enum). La variable y se inicializa en 1 y la variable d se inicializa con el valor Days.TUESDAY. La estructura switch evalúa d y salta directamente al caso que coincide: case TUESDAY. Allí se ejecuta la instrucción y += 10, por lo que y pasa a valer 11 (1 + 10). Ahora sí llegamos a la parte de la caída en cascada: Como no existe ninguna instrucción break; para detener el flujo, la ejecución continúa inmediatamente con el siguiente caso (case WEDNESDAY:). Se ejecuta y += 100, elevando el valor de y a 111 (11 + 100). En la acumulación final tenemos

que el flujo vuelve a caer al último caso (case THURSDAY:), donde se ejecuta y += 1000. El valor de y alcanza finalmente los 1111 (111 + 1000). Al salir del bloque switch, se imprime el valor final de y acumulado tras recorrer todas las ramas sin interrupción.

Pregunta número 13

Que se imprime en pantalla al ejecutar el siguiente código?

```
public class Test {
    public static void main(String... arguments) {
        final char a = 'A', d = 'D';
        char grade = 'B';
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

Seleccione una:

- a. A
- b. BF
- c. Compilation fails
- d. C
- e. ACF
- f. BCF
- g. B

Respuesta: En este ejercicio se combinan dos reglas importantes de la estructura switch en Java: los requisitos para las etiquetas case y el comportamiento de "fall-through" (caída en cascada). Las variables a y d se utilizan como etiquetas case dentro del bloque switch. En Java, los valores de los case deben ser obligatoriamente constantes en tiempo de compilación. Dado que a y d están declaradas con el modificador final e inicializadas inmediatamente con literales de caracteres, cumplen esta regla a la perfección, evitando un error de compilación. Por otro lado, la variable grade se evalúa con el valor 'B'. El flujo salta el case a: y aterriza exactamente en el case 'B':, ejecutando su instrucción para imprimir "B". Ahora sí llegamos al efecto cascada: Al igual que en ejercicios anteriores, vamos a notar la ausencia total de la palabra clave break; esto provoca que la ejecución no se detenga y continúe cayendo por todos los bloques subsecuentes sin importar si coinciden o no con el valor original. El flujo cae al case 'C': e imprime "C". Continúa cayendo, pasando por el case d: hasta llegar al case 'F', donde imprime "F". El resultado impreso secuencialmente es BCF.

Pregunta número 14

What is the result?

```
public class Test {
    public static void main(String... arguments) {
        final char a = 'A', d = 'D';
        char grade = 'B';
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
                break;
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

Seleccione una:

- a. B
- b. Compilation fails
- c. C
- d. A
- e. AC
- f. BC

Respuesta: Este ejercicio presenta una ligera variación del ejercicio anterior, evaluando nuevamente el uso de constantes y el control del flujo en cascada dentro de una estructura switch. Las variables a y d están declaradas como final e inicializadas con literales de caracteres, lo que las convierte en constantes en tiempo de compilación perfectamente válidas para las etiquetas case. El código compilará sin problemas. La variable a evaluar es grade, cuyo valor inicial es 'B'. El flujo de ejecución de la estructura salta directamente al punto de entrada

coincidente: case 'B'. Se ejecuta la instrucción de impresión, mostrando "B" en la consola. Al no existir una instrucción break; al final de este caso, ocurre el efecto de cascada ("fall-through") y la ejecución continúa forzosamente hacia el siguiente bloque inferior. El flujo ingresa al case 'C' y ejecuta su instrucción, imprimiendo "C" de forma adyacente. Inmediatamente después de esta impresión, el programa encuentra una instrucción break; explícita. Esto interrumpe exitosamente la cascada, deteniendo el flujo y forzando la salida inmediata del bloque switch antes de que pueda ejecutar los casos d o 'F'.

Pregunta número 15

Que muestra en pantalla el siguiente código?

```
public class Test {
    public static void main(String[] args) {
        int x = 0;
        while (x++ < 5) {
            // Empty loop body
        }
        String message = x > 5 ? "greater than" : "false";
        System.out.println(message + " - " + x);
    }
}
```

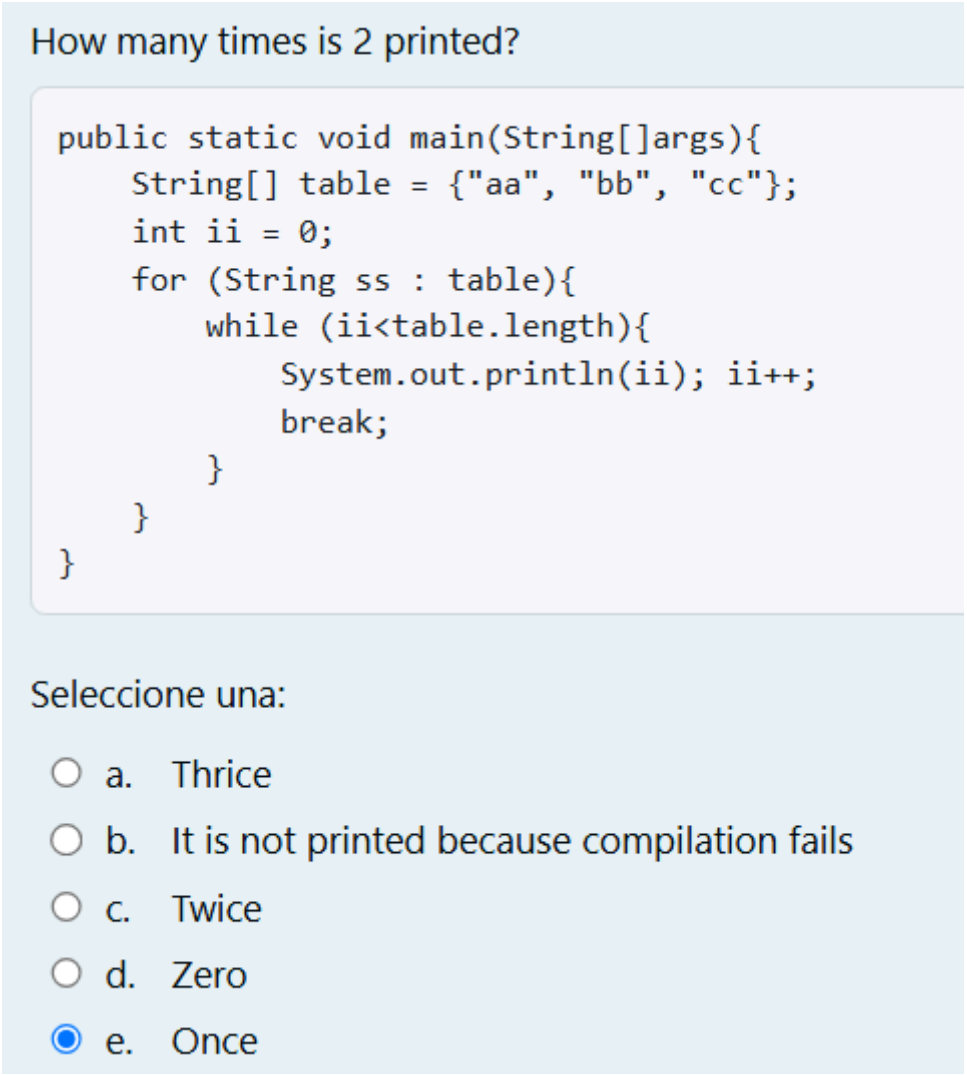
Seleccione una:

- a. false than - 6
- b. greater than - 6
- c. false than - 5
- d. Compilation fails.
- e. greater than - 5

Respuesta: En este ejercicio se está evaluando el comportamiento del operador de post-incremento (++) cuando se utiliza directamente dentro de la condición de un bucle while, así como la asignación mediante el operador ternario (? :): Vamos a realizar el seguimiento de la ejecución paso a paso para entender qué está pasando. La variable x se inicializa con el valor de 0. Al realizar la evaluación del bucle while (x++ < 5), tenemos que la característica clave del operador de

post-incremento es que primero devuelve el valor actual de la variable para evaluar la expresión y, después, incrementa su valor en 1. Cuando se evalúa 4 < 5, la condición es verdadera, y la variable x se incrementa a 5. En la siguiente iteración, la expresión evalúa el valor actual de x (que es 5). La condición 5 < 5 resulta ser falsa, lo que interrumpe el ciclo. Sin embargo, como el operador x++ ya se ejecutó en la evaluación, la variable x sufre un último incremento, pasando a valer 6 en la memoria. A continuación, se evalúa la instrucción String message = x > 5 ? "greater than" : "false". Dado que 6 > 5 es verdadero, a la variable message se le asigna la cadena "greater than". Finalmente, se concatena y se imprime el resultado de message + " - " + x, lo que se traduce en "greater than" + " - " + 6.

Pregunta número 16



Respuesta: En este ejercicio se está evaluando la interacción entre bucles anidados y la instrucción de salto break. Vamos a trazar la ejecución paso a paso. Empezamos con el arreglo table, que tiene 3 elementos ("aa", "bb", "cc"), por lo que su length es 3. La variable entera ii se inicializa en 0 fuera de los bucles. En la primera iteración del for (elemento "aa") se evalúa el while (0 < 3), lo cual es

verdadero. Se imprime 0 y ii se incrementa a 1. Inmediatamente se ejecuta la instrucción break, la cual interrumpe y finaliza únicamente el bucle interior (while). En la segunda iteración del for (elemento "bb") se evalúa el while (1 < 3), lo cual es verdadero. Se imprime 1 y ii se incrementa a 2. El break vuelve a interrumpir el while. En la tercera iteración del for (elemento "cc") se evalúa el while (2 < 3), lo cual es verdadero. Se imprime 2 y ii se incrementa a 3. El break interrumpe el while por última vez. El bucle for termina de recorrer el arreglo. Como podemos observar en la salida generada (0, 1, 2), el número 2 se imprime exactamente una vez durante la última iteración.

Pregunta número 17

Cual es el resultado?

```
public class Test {
    static boolean b;
    public static void main(String[] args) {
        int x = 0;
        if (b) {
            x = 1;
        } else if (b = false) {
            x = 2;
        } else if (b) {
            x = 3;
        } else {
            x = 4;
        }
        System.out.println("x=" + x);
    }
}
```

Seleccione una:

- a. x = 3
- b. x = 0
- c. Compilation fails
- d. x = 4
- e. x = 1

Respuesta: En este ejercicio se está evaluando el conocimiento sobre los valores por defecto de las variables de clase y la diferencia crítica entre los operadores de asignación e igualdad dentro de las expresiones condicionales. Primero se declara la variable estática b de tipo boolean a nivel de clase. Al no inicializarse de forma explícita, Java le asigna automáticamente su valor por defecto, que es false. A su vez, la variable local x se inicializa en 0. En la primera evaluación, al llegar a la primera condición if (b), el valor evaluado es false. El flujo de control salta este bloque y omite la asignación x = 1. La siguiente condición evalúa la expresión else if (b = false). Es fundamental notar el uso de un solo signo igual (=), que representa un operador de asignación, no el operador de igualdad relacional (==). Esta instrucción reasigna el valor false a la variable b y luego devuelve ese mismo valor como resultado booleano de la condición. Por lo tanto, esta evaluación también falla y se salta x = 2. En la tercera evaluación, la instrucción else if (b) vuelve a evaluar el valor actual de b, el cual sigue siendo false por la reasignación anterior. Se omite x = 3. Como absolutamente todas las condiciones if resultaron falsas, el programa entra en el bloque else final por descarte, ejecutando la instrucción x = 4. Al finalizar las estructuras de control, la instrucción System.out.println concatena y muestra el valor final de la variable en consola.

Pregunta número 18

Which one is valid as a replacement for foo?

```
public static void main(String[] args) {
    Boolean b1 = true;
    Boolean b2 = false;
    int i = 0;
    while (foo) { }
}
```

Seleccione una:

- a. b1.compareTo(b2)
- b. i = 1
- c. i == 2? -1:0
- d. "foo".equals("bar")

Respuesta: En este ejercicio se evalúa una regla estricta y fundamental del lenguaje Java: las condiciones dentro de las estructuras de control cíclicas y

condicionales (como el bucle while) deben evaluar obligatoriamente a un tipo boolean o a su clase envoltorio Boolean. Java no permite conversiones implícitas desde números u otros tipos hacia booleanos, a diferencia de lenguajes como C o C++. Vamos a analizar por qué las demás opciones fallan y la opción 'd' es la única correcta. Empecemos con el inciso a. b1.compareTo(b2): El método compareTo implementado en la clase Boolean no devuelve un valor booleano, sino un tipo int (típicamente 1, 0 o -1) para indicar el resultado de la comparación léxica/numérica. Usar un int como condición generará un error de compilación. Vamos con el inciso b. i = 1: Esta es una expresión de asignación. En Java, una asignación devuelve el valor asignado, que en este caso es el tipo primitivo int con valor 1. Al no ser booleano, el compilador lo rechaza. Seguimos con c. i == 2 ? -1 : 0: Aunque la primera parte (i == 2) es una evaluación booleana, el resultado final de toda la expresión ternaria será el entero -1 o el entero 0. Nuevamente, un tipo int es inválido para el bucle. Finalmente, tenemos el inciso ganador, que es el d. "foo".equals("bar"): El método equals() de la clase String compara el contenido de las cadenas y está diseñado para retornar estrictamente un valor booleano (true o false). En este caso particular devolverá false, lo que proporciona el tipo de dato correcto que el bucle while exige para compilar de forma exitosa.

Pregunta número 19

¿Que imprime en pantalla el código anterior?

```
public class Test {
    static int co = 0;
    public static void main(String[] args) {
        int x = 1;
        for (test('1'); test('2') && (x <= 2); test('3')) {
            x++;
            test('4');
        }
        System.out.println(" The co value = " + co);
    }
    static boolean test(char num) {
        System.out.print("" + num); co++; return true;
    }
}
```

Seleccione una:

- a. 12432432 The co value = 8
- b. 1234 The co value = 4
- c. 14231423 The co value = 8
- d. 12341234 The co value = 8
- e. 12342342 The co value = 8
- f. Compilation fails

Respuesta: El código evalúa un bucle for con llamadas a un método test en cada una de sus partes (inicialización, condición y actualización). El método test imprime el carácter que recibe, incrementa el contador co y siempre devuelve true. Además, la condición utiliza el operador de cortocircuito &&. Vamos paso a paso por la ejecución para entender qué está sucediendo: El primer paso es la inicialización del bucle for: Se ejecuta test('1'), se imprime 1 y co se incrementa a 1. Primera iteración: Condición: test('2') && (x <= 2). Se evalúa primero test('2'). Imprime 2 y co pasa a 2. Devuelve true. Como el lado izquierdo es true, se evalúa el lado derecho (x <= 2). Como x es 1, la condición es true. El resultado total es true, así que entra al cuerpo. Cuerpo del bucle: x++ (x pasa a 2) y test('4'). Imprime 4 y co pasa a 3.

Actualización con test('3'). Imprime 3 y co pasa a 4. Segunda iteración: Condición: test('2') && (x <= 2). Se evalúa test('2'). Imprime 2 y co pasa a 5. Devuelve true. Se evalúa (x <= 2). Como x es 2, la condición es true. Se entra al cuerpo. Cuerpo del bucle: x++ (x pasa a 3) y test('4'). Imprime 4 y co pasa a 6. Actualización con test('3'). Imprime 3 y co pasa a 7. Vamos con la iteración final: Condición: test('2') && (x <= 2). Se evalúa test('2'). Imprime 2 y co pasa a 8. Devuelve true. Se evalúa (x <= 2). Como x es 3, la condición es false. El bucle termina. Algo a notar es que test('2') ya se ejecutó e imprimió antes de que se evaluara el lado derecho. Después del bucle: Se ejecuta System.out.println("The co value = " + co). Imprime: The co value = 8. Al concatenar todo lo impreso por el método test (1, 2, 4, 3, 2, 4, 3, 2) y la impresión final, obtenemos: 12432432 The co value = 8.

Pregunta número 20

What values of x, y, z will produce the following result?

```
1 2 3 4
1 2 3 4
1 2 3 4
---
1 2 3 4
---
```

```
public static void main(String[] args){
    //insert code here
    int j = 0, k = 0;
    for (int i = 0; i<x; i++){
        do{
            k = 0;
            while (k<z){
                k++;
                System.out.print(k + " ");
            }
            System.out.println(" ");
            j++;
        }while (j<y);
        System.out.println("---");
    }
}
```

Seleccione una:

- a. int x = 4, y = 3, z = 2;
- b. int x = 2, y = 3, z = 3;
- c. int x = 3, y = 2, z = 3;
- d. int x = 2, y = 3, z = 4;
- e. int x = 4, y = 2, z = 3;

Respuesta: Para determinar los valores correctos, debemos analizar cómo las variables x, y y z controlan los bucles y cómo se comporta la variable j. Primero tenemos que z controla el bucle interior while (k < z): Este bucle es el encargado de imprimir la secuencia de números. La salida esperada muestra la secuencia 1 2 3 4. Como el bucle incrementa k y lo imprime mientras k < z, para que se impriman hasta el 4, el valor de z debe ser 4. La x controla el bucle exterior for (int i = 0; i < x; i++): Este bucle determina cuántas veces se repite todo el bloque interior. La salida muestra dos secciones separadas por la línea ---. Esto indica que el bucle exterior se ejecuta 2 veces. Por lo tanto, x debe ser 2. La y controla el bucle do-while (j < y): Aquí está la clave, ya que la variable j se inicializa a 0 fuera del bucle for. Esto significa que j no se reinicia en cada iteración del bucle exterior. Primera iteración

del for (i = 0): j empieza en 0. El bucle do-while se ejecuta repetidamente. La salida muestra 3 líneas de 1 2 3 4 antes del primer ---. Esto significa que el bucle se ejecutó 3 veces. Por lo tanto, y debe ser 3. Al terminar, j vale 3. Segunda iteración del for (i = 1): j no se reinicia, sigue valiendo 3. El bucle do-while se ejecuta al menos una vez (por ser do-while). Imprime una línea de 1 2 3 4. Luego evalúa la condición while (j < y), es decir, while (3 < 3). Como es falso, el bucle termina. La salida muestra exactamente 1 línea de 1 2 3 4 antes del segundo ---. Esto coincide perfectamente con el inciso d.